30-DataFrame Handling, Grouping, and Data Cleaning in pandas

Introduction

- The session focuses on handling and manipulating data in pandas DataFrames, emphasizing shaping, concatenation, merging, reshaping (pivot and melt), grouping, and handling missing data.

Concatenation and Merging DataFrames

Concatenation (pd.concat)

- Combines two DataFrames vertically, appending one after the other.
- Common columns appear once; non-common columns have `None` or `NaN` for missing values in concatenated frames.
- No requirement for shared columns; columns unique to each DataFrame remain.
- The default index from both DataFrames is preserved unless `ignore_index=True` is used.

Merging (pd.merge)

- Combines two DataFrames based on one or more common columns (keys).
- Requires at least one common column to perform merge; otherwise, results in error.
- Can specify the merge column(s) using the `on` attribute.

• If multiple columns are common, merges on all unless otherwise specified.

• Handles duplicate key values correctly, generating appropriate combinations.

• Supports different types of joins:

- **Left Join:**
All data from the left DataFrame and matching data from the right; non-matching right data is filled with nulls.

- **Right Join:**
All data from the right DataFrame and matching data from the left; non-matching left data is filled with nulls.

- **Outer Join:** All data from both DataFrames; non-matching entries filled with nulls.

- Join type is specified with the `how` parameter (`left`, `right`, `outer`).

Reshaping DataFrames: Pivot and Melt

Pivot Tables (pivot)

• Used for restructuring data, reducing number of rows, or summarizing data by groups (similar to Excel pivot tables).

• Example: Converting long-form sales data (e.g., types of bags delivered in regions) into a wide format with regions as rows and types as columns.

• Reduces data redundancy; enables easier analysis by summarizing or segmenting data.

• Requires specifying which column to use as index (rows) and which to use as columns.

Melting (pd.melt)

- Opposite of pivot; transforms wide data (many columns) into long-form (fewer columns, more rows).
- Useful for consolidating multiple value columns into a single column and creating categorical key columns.
- Requires specifying which variable to keep fixed (e.g., months repeated per city), and the variable/value column names can be customized.
- Allows easy filtering and analysis of data across a categorical dimension.

Grouping Data (groupby)

- Used to categorize or segment data for analysis (e.g., by region, age group, or country of origin).
- Example: Grouping car data by origin (US, Europe, Japan) for statistical summaries.
- Syntax: `groupby(column)`, returns group objects; specific groups can be accessed with `get_group(value)`.
- After grouping, aggregate functions like `max()`, `mean()`, etc., can be applied to each group.
- Iterating over groups yields a tuple (group value, group DataFrame).

Handling Missing Data

- **Detecting Missing Values:**
  - `.isna()` returns Boolean dataframe of missing (null) values.

- `.notna()` returns Boolean dataframe of non-missing values.

- Summing Boolean results provides total count of missing/non-missing values per column.

• **Filling Missing Values:**

- `.fillna(value)` replaces nulls with a specified value; can use a dictionary to replace different columns with distinct values.

- Forward fill (`method='ffill'`) replaces nulls with preceding non-null values.

- Backward fill (`method='bfill'`) replaces nulls with the next row's value.

- Interpolation (`.interpolate()`) fills missing numeric values using linear or specified methods; not suitable for string data.

• **Read Options:**

- Can read date columns as datetime type using `parse_dates` on file import.

- Can set any column as index with `set_index(column, inplace=True)`.

• Careful consideration is needed when cleaning missing data; removing all rows with nulls can drastically shrink data set and is often unsuitable.

Q&A and Assignment Discussion

• Students discussed issues with assignment questions, particularly relating to image data slicing and dimension indexing.

• Examples for slicing multi-dimensional arrays (e.g., images with height, width, channel) and discussion on how pandas and numpy handle these.

- Clarifications provided for selecting specific channels and image patches.

- Assignment logistics: datasets are uploaded, and further practice on matplotlib and seaborn will continue in future sessions.